

Here's a breakdown of the SOLID principles with Java examples for each:

1. Single Responsibility Principle (SRP)

A class should have only one reason to change, meaning it should only have one job or responsibility.

Example:

```
java
// Violation of SRP
class Employee {
    public void calculateSalary() {
        // salary calculation logic
    }

    public void generateReport() {
        // report generation logic
    }
}

// Applying SRP
class Employee {
    // Employee related attributes and methods
}

class SalaryCalculator {
    public void calculateSalary(Employee employee) {
        // salary calculation logic
    }
}

class ReportGenerator {
    public void generateReport(Employee employee) {
        // report generation logic
    }
}
```

2. Open/Closed Principle (OCP)

Classes should be open for extension but closed for modification.

Example:

java

```
// Without OCP - modifying the class for every new shape
class AreaCalculator {
    public double calculateArea(Object shape) {
        if (shape instanceof Circle) {
            Circle circle = (Circle) shape;
            return Math.PI * circle.radius * circle.radius;
        } else if (shape instanceof Rectangle) {
            Rectangle rectangle = (Rectangle) shape;
            return rectangle.length * rectangle.width;
        }
        return 0;
    }
}

// Applying OCP - extend behavior via inheritance
abstract class Shape {
    abstract double calculateArea();
}

class Circle extends Shape {
    double radius;
    public Circle(double radius) {
        this.radius = radius;
    }
    public double calculateArea() {
        return Math.PI * radius * radius;
    }
}

class Rectangle extends Shape {
    double length, width;
    public Rectangle(double length, double width) {
        this.length = length;
        this.width = width;
    }
    public double calculateArea() {
        return length * width;
    }
}
```

```
class AreaCalculator {
    public double calculateArea(Shape shape) {
        return shape.calculateArea();
    }
}
```

3. Liskov Substitution Principle (LSP)

Objects of a superclass should be replaceable with objects of a subclass without affecting the correctness of the program.

Example:

```
java
// Violation of LSP - subclass alters expected behavior
class Bird {
    public void fly() {
        System.out.println("Flying");
    }
}

class Ostrich extends Bird {
    @Override
    public void fly() {
        throw new UnsupportedOperationException("Ostrich can't
fly");
    }
}

// Applying LSP - better class design
class Bird {
    public void eat() {
        System.out.println("Eating");
    }
}

class FlyingBird extends Bird {
    public void fly() {
        System.out.println("Flying");
    }
}
```

```
class Sparrow extends FlyingBird {
    // Sparrow can fly
}

class Ostrich extends Bird {
    // Ostrich cannot fly but can still be a bird
}
```

4. Interface Segregation Principle (ISP)

Clients should not be forced to implement interfaces they do not use.

Example:

```
java
// Violation of ISP - forcing all animals to implement fly method
interface Animal {
    void eat();
    void fly();
}

class Dog implements Animal {
    public void eat() {
        // eating logic
    }
    public void fly() {
        // Dog can't fly, but must implement fly method
    }
}

// Applying ISP - split interfaces
interface Eater {
    void eat();
}

interface Flyer {
    void fly();
}

class Dog implements Eater {
    public void eat() {
```

```

        // eating logic
    }
}

class Bird implements Eater, Flyer {
    public void eat() {
        // eating logic
    }

    public void fly() {
        // flying logic
    }
}

```

5. Dependency Inversion Principle (DIP)

High-level modules should not depend on low-level modules. Both should depend on abstractions. Abstractions should not depend on details; details should depend on abstractions.

Example:

```

java
// Without DIP - high-level class depends on low-level class
class Light {
    public void turnOn() {
        System.out.println("Light is on");
    }
}

class Switch {
    private Light light;

    public Switch() {
        this.light = new Light();
    }

    public void operate() {
        light.turnOn();
    }
}

```

```
// Applying DIP - high-level class depends on abstraction
interface Switchable {
    void turnOn();
}

class Light implements Switchable {
    public void turnOn() {
        System.out.println("Light is on");
    }
}

class Fan implements Switchable {
    public void turnOn() {
        System.out.println("Fan is on");
    }
}

class Switch {
    private Switchable device;

    public Switch(Switchable device) {
        this.device = device;
    }

    public void operate() {
        device.turnOn();
    }
}
```

Each of these principles ensures that the code remains maintainable, scalable, and flexible over time.

Here We can discuss the use of Design patterns Why we Need this

Before Strategy Pattern

Without using the Strategy Pattern, you might have a single class with conditional logic to handle different behaviors. This results in code that's difficult to maintain and extend.

java

```
class PaymentProcessor {  
    public void processPayment(String paymentType) {  
        if (paymentType.equals("creditCard")) {  
            System.out.println("Processing credit card payment...");  
        } else if (paymentType.equals("paypal")) {  
            System.out.println("Processing PayPal payment...");  
        } else {  
            System.out.println("Unknown payment method.");  
        }  
    }  
}
```

This `PaymentProcessor` class handles multiple types of payment. If you need to add more payment methods, you'd have to modify the class, violating the **Open-Closed Principle**.

After Strategy Pattern

Using the Strategy Pattern, you can encapsulate each payment method into its own class and make the system easier to extend.

Step 1: Create a Strategy Interface

java

Copy code

```
interface PaymentStrategy {  
    void processPayment();  
}
```

Step 2: Implement Concrete Strategies

java

```
class CreditCardPayment implements PaymentStrategy {  
  
    @Override  
  
    public void processPayment() {  
  
        System.out.println("Processing credit card payment...");  
  
    }  
  
}
```

```
class PayPalPayment implements PaymentStrategy {  
  
    @Override  
  
    public void processPayment() {  
  
        System.out.println("Processing PayPal payment...");  
  
    }  
  
}
```

Step 3: Create a Context Class

Java

```
class PaymentProcessor {  
  
    private PaymentStrategy strategy;  
  
  
    public PaymentProcessor(PaymentStrategy strategy) {  
  
        this.strategy = strategy;  
  
    }  
  
  
    public void processPayment() {
```



```
        strategy.processPayment();
    }
}
```

Step 4: Use the Strategy in Client Code

java

```
public class Main {

    public static void main(String[] args) {

        PaymentProcessor creditCardProcessor = new
PaymentProcessor(new CreditCardPayment());

        creditCardProcessor.processPayment();

        PaymentProcessor paypalProcessor = new PaymentProcessor(new
PayPalPayment());

        paypalProcessor.processPayment();

    }

}
```

Benefits of Using the Strategy Pattern

1. **Open-Closed Principle:** You can add new payment strategies without modifying the `PaymentProcessor` class.
2. **Separation of Concerns:** Each payment method has its own class, making the code easier to maintain.
3. **Flexibility:** You can dynamically change the strategy at runtime if needed.

In this example, you've separated the behavior of different payment methods into separate classes and used the `PaymentStrategy` interface to switch between them.

Before Factory Pattern

Without using the Factory Pattern, object creation is done directly in the client code. This can make the code tightly coupled and difficult to maintain if object creation becomes complex or changes over time.

Example

java

```
class Circle {

    public void draw() {

        System.out.println("Drawing a circle.");

    }

}

class Rectangle {

    public void draw() {

        System.out.println("Drawing a rectangle.");

    }

}

public class ShapeClient {

    public static void main(String[] args) {

        Circle circle = new Circle();

        circle.draw();

        Rectangle rectangle = new Rectangle();
```

```
        rectangle.draw();
    }
}
```

In this case, the client (`ShapeClient`) is responsible for creating `Circle` and `Rectangle` objects. If you need to add a new shape (e.g., `Triangle`), you would need to modify the client code. This violates the **Open-Closed Principle**.

After Factory Pattern

The Factory Pattern abstracts the object creation process, allowing for flexibility and decoupling the client from the specific classes that are being instantiated.

Step 1: Create a Common Interface

```
java

interface Shape {

    void draw();

}
```

Step 2: Implement Concrete Classes

```
java

class Circle implements Shape {

    @Override

    public void draw() {

        System.out.println("Drawing a circle.");

    }

}
```

```
class Rectangle implements Shape {
```

```
@Override  
  
public void draw() {  
    System.out.println("Drawing a rectangle.");  
}  
}
```

```
class Triangle implements Shape {  
  
    @Override  
  
    public void draw() {  
        System.out.println("Drawing a triangle.");  
    }  
}
```

Step 3: Create a Factory Class

java

```
class ShapeFactory {  
  
    public Shape getShape(String shapeType) {  
        if (shapeType == null) {  
            return null;  
        }  
  
        if (shapeType.equalsIgnoreCase("CIRCLE")) {  
            return new Circle();  
        } else if (shapeType.equalsIgnoreCase("RECTANGLE")) {  
            return new Rectangle();  
        } else if (shapeType.equalsIgnoreCase("TRIANGLE")) {
```

```
        return new Triangle();
    }

    return null;
}
}
```

Step 4: Use the Factory in Client Code

java

```
public class ShapeClient {

    public static void main(String[] args) {

        ShapeFactory shapeFactory = new ShapeFactory();

        Shape shape1 = shapeFactory.getShape("CIRCLE");

        shape1.draw();

        Shape shape2 = shapeFactory.getShape("RECTANGLE");

        shape2.draw();

        Shape shape3 = shapeFactory.getShape("TRIANGLE");

        shape3.draw();

    }

}
```

Benefits of Using the Factory Pattern

1. **Decoupling:** The client code is not tightly coupled to the specific shape classes (`Circle`, `Rectangle`, `Triangle`). This makes the code more flexible and easier to maintain.
2. **Open-Closed Principle:** New shapes can be added without modifying the client code. You simply add a new shape class and update the `ShapeFactory` without changing how objects are created in the client.
3. **Reusability:** The factory can be reused to create instances of different objects based on input parameters.

In this example, the Factory Pattern encapsulates object creation, providing flexibility in adding new shapes while keeping the client code clean and maintainable.

Before Decorator Pattern

Without the Decorator Pattern, if you want to extend functionality of a class, you often end up creating multiple subclasses that can lead to a large inheritance hierarchy. This can make the code hard to maintain and less flexible.

Example

Let's say you have a `Coffee` class, and you want to add different flavors like "Milk" and "Sugar". Without the decorator pattern, you would handle this by creating multiple subclasses.

java

```
class Coffee {  
  
    public String getDescription() {  
  
        return "Plain Coffee";  
  
    }  
  
  
    public double cost() {  
  
        return 5.0;  
  
    }  
  
}
```

```
class MilkCoffee extends Coffee {

    @Override

    public String getDescription() {

        return "Coffee with Milk";

    }


    @Override

    public double cost() {

        return 6.0;

    }

}


class MilkSugarCoffee extends MilkCoffee {

    @Override

    public String getDescription() {

        return "Coffee with Milk and Sugar";

    }


    @Override

    public double cost() {

        return 7.0;

    }

}
```

```

public class CoffeeShop {

    public static void main(String[] args) {

        Coffee milkSugarCoffee = new MilkSugarCoffee();

        System.out.println(milkSugarCoffee.getDescription() + "
costs $" + milkSugarCoffee.cost());

    }

}

```

In this case, we have a tight inheritance hierarchy. Adding more combinations of coffee types (e.g., with chocolate, vanilla, etc.) would result in even more subclasses.

After Decorator Pattern

The Decorator Pattern allows you to dynamically add behavior to objects without creating a complex inheritance structure. It involves wrapping the original object with decorators that add functionality.

Step 1: Create a Common Interface or Abstract Class

java

```

abstract class Coffee {

    public abstract String getDescription();

    public abstract double cost();

}

```

Step 2: Implement the Base Concrete Class

java

```

class PlainCoffee extends Coffee {

    @Override

    public String getDescription() {

        return "Plain Coffee";

    }

}

```



```
    }

    @Override
    public double cost() {
        return 5.0;
    }
}
```

Step 3: Create the Decorator Abstract Class

```
java
abstract class CoffeeDecorator extends Coffee {
    protected Coffee coffee;

    public CoffeeDecorator(Coffee coffee) {
        this.coffee = coffee;
    }

    public abstract String getDescription();
}
```

Step 4: Implement Concrete Decorators

```
Java
class MilkDecorator extends CoffeeDecorator {
    public MilkDecorator(Coffee coffee) {
        super(coffee);
    }
}
```

```

    }

    @Override
    public String getDescription() {
        return coffee.getDescription() + ", Milk";
    }

    @Override
    public double cost() {
        return coffee.cost() + 1.0;
    }
}

class SugarDecorator extends CoffeeDecorator {
    public SugarDecorator(Coffee coffee) {
        super(coffee);
    }

    @Override
    public String getDescription() {
        return coffee.getDescription() + ", Sugar";
    }

    @Override

```

```
        public double cost() {  
            return coffee.cost() + 0.5;  
        }  
    }  
}
```

Step 5: Use Decorators in Client Code

java

```
public class CoffeeShop {  
    public static void main(String[] args) {  
        Coffee coffee = new PlainCoffee();  
        System.out.println(coffee.getDescription() + " costs $" +  
coffee.cost());  
  
        Coffee milkCoffee = new MilkDecorator(new PlainCoffee());  
        System.out.println(milkCoffee.getDescription() + " costs $"  
+ milkCoffee.cost());  
  
        Coffee milkSugarCoffee = new SugarDecorator(new  
MilkDecorator(new PlainCoffee()));  
        System.out.println(milkSugarCoffee.getDescription() + "  
costs $" + milkSugarCoffee.cost());  
    }  
}
```

Benefits of Using the Decorator Pattern

1. **Flexibility:** The decorator pattern allows you to add or remove responsibilities to objects dynamically. This gives you greater flexibility than inheritance, where behaviors are fixed.
2. **Reusability:** Decorators can be reused for different objects, and you can mix and match them as needed without needing to create multiple subclasses.
3. **Single Responsibility:** Each decorator class has a single responsibility (e.g., `MilkDecorator` adds milk). This makes the code cleaner and more maintainable.

In this example, instead of creating multiple subclasses to handle different combinations of coffee types (e.g., "Coffee with Milk and Sugar"), we use decorators to dynamically add the behavior at runtime. This approach avoids a complex inheritance hierarchy and makes the code more flexible and scalable.

Before Observer Pattern

Without using the Observer Pattern, the `WeatherStation` class is directly responsible for notifying the display devices (like a `CurrentConditionsDisplay`) when there is a change in weather data. This creates tight coupling between the `WeatherStation` and the displays, making it hard to add or remove display devices without modifying the `WeatherStation` class itself.

Example

java

```
class WeatherStation {  
  
    private float temperature;  
  
    private float humidity;  
  
    private float pressure;  
  
    private CurrentConditionsDisplay currentConditionsDisplay;  
  
    public WeatherStation(CurrentConditionsDisplay  
currentConditionsDisplay) {  
  
        this.currentConditionsDisplay = currentConditionsDisplay;  
  
    }  
}
```

```
    public void setMeasurements(float temperature, float humidity,
float pressure) {

        this.temperature = temperature;

        this.humidity = humidity;

        this.pressure = pressure;

        updateDisplay();

    }
```

```
    private void updateDisplay() {

        currentConditionsDisplay.update(temperature, humidity,
pressure);

    }

}
```

```
class CurrentConditionsDisplay {

    public void update(float temperature, float humidity, float
pressure) {

        System.out.println("Current conditions: " + temperature + "F
degrees, " +

            humidity + "% humidity, and " + pressure + "
pressure.");

    }

}
```

```
public class WeatherStationApp {

    public static void main(String[] args) {
```

```

        CurrentConditionsDisplay currentDisplay = new
CurrentConditionsDisplay();

        WeatherStation weatherStation = new
WeatherStation(currentDisplay);

        // Changing weather data

        weatherStation.setMeasurements(80, 65, 30.4f);

        weatherStation.setMeasurements(82, 70, 29.2f);

    }
}

```

Problems:

- The `WeatherStation` is tightly coupled with the `CurrentConditionsDisplay`.
- If you need to add more displays (like `ForecastDisplay`, `StatisticsDisplay`), you have to modify the `WeatherStation` class.
- No flexibility in dynamically adding or removing display devices.

After Observer Pattern

By using the **Observer Pattern**, we decouple the `WeatherStation` (the subject) from the display devices (observers). Each observer can register/unregister itself with the `WeatherStation`, and the station will notify all registered observers whenever the weather data changes. This makes the system more flexible and scalable.

Step 1: Create the Subject (WeatherStation) Interface

java

```

interface Subject {

    void registerObserver(Observer o);

    void removeObserver(Observer o);

    void notifyObservers();
}

```

```
}
```

Step 2: Create the Observer Interface

```
java
```

```
interface Observer {  
    void update(float temperature, float humidity, float pressure);  
}
```

Step 3: Implement the WeatherStation as the Concrete Subject

```
java
```

```
import java.util.ArrayList;  
  
class WeatherStation implements Subject {  
    private ArrayList<Observer> observers;  
    private float temperature;  
    private float humidity;  
    private float pressure;  
  
    public WeatherStation() {  
        observers = new ArrayList<>();  
    }  
  
    @Override  
    public void registerObserver(Observer o) {  
        observers.add(o);  
    }  
}
```

```

    }

    @Override
    public void removeObserver(Observer o) {
        observers.remove(o);
    }

    @Override
    public void notifyObservers() {
        for (Observer observer : observers) {
            observer.update(temperature, humidity, pressure);
        }
    }

    public void setMeasurements(float temperature, float humidity,
float pressure) {
        this.temperature = temperature;
        this.humidity = humidity;
        this.pressure = pressure;
        notifyObservers(); // Notify all observers of the change
    }
}

```

Step 4: Create Concrete Observers

java


```
class CurrentConditionsDisplay implements Observer {

    @Override

    public void update(float temperature, float humidity, float
pressure) {

        System.out.println("Current conditions: " + temperature + "F
degrees, " +

            humidity + "% humidity, and " + pressure + "
pressure.");

    }

}
```

```
class ForecastDisplay implements Observer {

    @Override

    public void update(float temperature, float humidity, float
pressure) {

        System.out.println("Forecast: More of the same.");

    }

}
```

Step 5: Use the Observer Pattern in the Client Code

java

```
public class WeatherStationApp {

    public static void main(String[] args) {

        WeatherStation weatherStation = new WeatherStation();

        CurrentConditionsDisplay currentDisplay = new
CurrentConditionsDisplay();
```

```
ForecastDisplay forecastDisplay = new ForecastDisplay();

weatherStation.registerObserver(currentDisplay);

weatherStation.registerObserver(forecastDisplay);

// Changing weather data

weatherStation.setMeasurements(80, 65, 30.4f);

weatherStation.setMeasurements(82, 70, 29.2f);

}

}
```

Benefits of Using the Observer Pattern

1. **Loose Coupling:** The `WeatherStation` (subject) and the display devices (`CurrentConditionsDisplay`, `ForecastDisplay`, etc.) are loosely coupled. The subject doesn't need to know the specifics of each observer, making the system more maintainable.
2. **Scalability:** You can easily add new display devices by simply creating new observers. The subject doesn't need to change.
3. **Flexibility:** Observers can be added or removed dynamically during runtime, providing flexibility in how the system behaves.

In this example, the **Observer Pattern** is used to manage a one-to-many relationship between the `WeatherStation` (subject) and the displays (observers). Each observer can independently subscribe/unsubscribe to the subject's updates, making the code more flexible and easier to maintain.

The **Mediator** and **Command** patterns are behavioral design patterns that help decouple components in a system and manage interactions in a flexible way. Let me explain both patterns with examples.

1. Mediator Pattern

Purpose: The Mediator pattern reduces the dependencies between components (also called "colleagues") by introducing a mediator that controls the communication between them. This helps in avoiding tight coupling and makes the system more flexible and maintainable.

Example: Chat Room

In a chat room system, each user sends messages to other users. If each user were to communicate directly with others, the system would become very complex. Instead, we introduce a **ChatRoom (Mediator)** that manages communication between users.

Code Example (Java):

Java

```
// Mediator Interface
```

```
interface ChatRoomMediator {  
  
    void showMessage(User user, String message);  
  
}
```

```
// Concrete Mediator
```

```
class ChatRoom implements ChatRoomMediator {  
  
    @Override  
  
    public void showMessage(User user, String message) {  
  
        System.out.println(user.getName() + ": " + message);  
  
    }  
  
}
```

```
// Colleague
```

```
class User {
```

```
private String name;

private ChatRoomMediator chatRoomMediator;

public User(String name, ChatRoomMediator chatRoomMediator) {

    this.name = name;

    this.chatRoomMediator = chatRoomMediator;

}

public String getName() {

    return name;

}

public void sendMessage(String message) {

    chatRoomMediator.showMessage(this, message);

}

}

// Client

public class MediatorPatternExample {

    public static void main(String[] args) {

        ChatRoomMediator chatRoom = new ChatRoom();

        User john = new User("John", chatRoom);

        User jane = new User("Jane", chatRoom);

    }

}
```

```
        john.sendMessage("Hi, Jane!");

        jane.sendMessage("Hello, John!");
    }
}
```

Output:

John: Hi, Jane!

Jane: Hello, John!

Here, the `ChatRoom` acts as the **Mediator** and manages the communication between different users, reducing direct dependencies between them.

2. Command Pattern

Purpose: The Command pattern turns requests or operations into objects, which allows us to parameterize other objects with different requests, delay execution, or support undoable operations.

Example: Remote Control for Devices

Imagine you have a universal remote control that can turn on/off devices like a TV, stereo, and lights. The remote control doesn't need to know the details of how each device works; it only needs to send commands.

Code Example (Java):

```
// Command Interface

interface Command {

    void execute();

}

// Receiver
```

```
class Light {

    public void turnOn() {

        System.out.println("Light is ON");

    }

    public void turnOff() {

        System.out.println("Light is OFF");

    }

}

// Concrete Command for turning on the light
class TurnOnLightCommand implements Command {

    private Light light;

    public TurnOnLightCommand(Light light) {

        this.light = light;

    }

    @Override

    public void execute() {

        light.turnOn();

    }

}
```

```
// Concrete Command for turning off the light
class TurnOffLightCommand implements Command {
    private Light light;

    public TurnOffLightCommand(Light light) {
        this.light = light;
    }

    @Override
    public void execute() {
        light.turnOff();
    }
}
```

```
// Invoker
class RemoteControl {
    private Command command;

    public void setCommand(Command command) {
        this.command = command;
    }

    public void pressButton() {
        command.execute();
    }
}
```

```
    }  
}  
  
// Client  
  
public class CommandPatternExample {  
    public static void main(String[] args) {  
        Light livingRoomLight = new Light();  
        Command turnOn = new TurnOnLightCommand(livingRoomLight);  
        Command turnOff = new TurnOffLightCommand(livingRoomLight);  
  
        RemoteControl remote = new RemoteControl();  
  
        // Turn on the light  
        remote.setCommand(turnOn);  
        remote.pressButton();  
  
        // Turn off the light  
        remote.setCommand(turnOff);  
        remote.pressButton();  
    }  
}
```

Output:

Light is ON

Light is OFF

In this example:

- The **Light** is the **Receiver**.
 - **TurnOnLightCommand** and **TurnOffLightCommand** are **Command** implementations.
 - The **RemoteControl** is the **Invoker** that executes commands.
-

Key Differences:

- **Mediator**: Focuses on reducing direct communication between objects by introducing a mediator that controls interaction.
- **Command**: Encapsulates a request as an object, allowing it to be parameterized, delayed, or even undone.

Both patterns decouple the system components, but the **Mediator** focuses on managing interactions, while the **Command** focuses on encapsulating actions.

Example Scenario:

Let's say we have a complex system for a computer startup process. The system involves multiple subsystems such as the CPU, Memory, and HardDrive. Without a facade, the client needs to interact with each subsystem directly. After applying the facade pattern, the client interacts with a single interface that simplifies the interaction with the subsystems.

Before Applying Facade Pattern:

In this approach, the client needs to interact directly with multiple subsystems.

Code Example (Java):

```
java
```

```
// Subsystem 1: CPU
```

```
class CPU {
```

```
    public void freeze() {
```

```
        System.out.println("Freezing CPU...");
```

```
    }
```

```
        public void jump(long position) {

            System.out.println("Jumping to position " + position);

        }

        public void execute() {

            System.out.println("Executing CPU commands...");

        }

    }

// Subsystem 2: Memory

class Memory {

    public void load(long position, byte[] data) {

        System.out.println("Loading data at position " + position);

    }

}

// Subsystem 3: HardDrive

class HardDrive {

    public byte[] read(long lba, int size) {

        System.out.println("Reading data from LBA " + lba);

        return new byte[size];

    }

}
```

```
// Client

public class ComputerStartupWithoutFacade {

    public static void main(String[] args) {

        CPU cpu = new CPU();

        Memory memory = new Memory();

        HardDrive hardDrive = new HardDrive();


        // Client must handle multiple subsystems to start the
computer

        cpu.freeze();

        byte[] bootData = hardDrive.read(100, 512);

        memory.load(100, bootData);

        cpu.jump(100);

        cpu.execute();

    }

}
```

Output:

Freezing CPU...

Reading data from LBA 100

Loading data at position 100

Jumping to position 100

Executing CPU commands...

Problem:

The client has to interact with every subsystem individually, which makes the code complex and difficult to maintain. Every time a new subsystem or a modification is introduced, the client needs to change accordingly.

After Applying the Facade Pattern:

We introduce a **Facade** class that provides a simpler interface for the client. Now, the client only interacts with the facade to start the computer, and the facade handles all the interactions with the subsystems.

Code Example (Java):

```
java
```

```
// Subsystem 1: CPU (Unchanged)
```

```
class CPU {
```

```
    public void freeze() {
```

```
        System.out.println("Freezing CPU...");
```

```
    }
```

```
    public void jump(long position) {
```

```
        System.out.println("Jumping to position " + position);
```

```
    }
```

```
    public void execute() {
```

```
        System.out.println("Executing CPU commands...");
```

```
    }
```

```
}
```

```
// Subsystem 2: Memory (Unchanged)
```

```
class Memory {  
    public void load(long position, byte[] data) {  
        System.out.println("Loading data at position " + position);  
    }  
}
```

// Subsystem 3: HardDrive (Unchanged)

```
class HardDrive {  
    public byte[] read(long lba, int size) {  
        System.out.println("Reading data from LBA " + lba);  
        return new byte[size];  
    }  
}
```

// Facade

```
class ComputerFacade {  
    private CPU cpu;  
    private Memory memory;  
    private HardDrive hardDrive;  
  
    public ComputerFacade() {  
        this.cpu = new CPU();  
        this.memory = new Memory();  
        this.hardDrive = new HardDrive();  
    }  
}
```

```

    }

    public void start() {
        System.out.println("Starting the computer...");
        cpu.freeze();
        byte[] bootData = hardDrive.read(100, 512);
        memory.load(100, bootData);
        cpu.jump(100);
        cpu.execute();
    }
}

// Client
public class ComputerStartupWithFacade {
    public static void main(String[] args) {
        ComputerFacade computer = new ComputerFacade();

        // Now, client uses the facade to start the computer
        computer.start();
    }
}

```

Output:

```
Starting the computer...
```

Freezing CPU...

Reading data from LBA 100

Loading data at position 100

Jumping to position 100

Executing CPU commands...

Benefits of Facade Pattern:

1. **Simplified Interface:** The client no longer has to deal with the complexities of the individual subsystems (CPU, Memory, and HardDrive). Instead, it interacts only with the ComputerFacade.
2. **Decoupling:** Changes in subsystems (like adding new methods or modifying existing ones) do not affect the client. The facade shields the client from these changes.
3. **Better Maintenance:** The system becomes easier to maintain, as the complex interactions between subsystems are encapsulated in the facade.

Purpose: The Visitor pattern lets you add further operations to objects without modifying their structures. It allows you to define new operations on a set of objects by placing the operations in a separate object, called a visitor. This promotes flexibility and separation of concerns.

Example: Shape Visitor

Imagine a drawing system with different shapes. You want to perform different operations (e.g., calculating area, drawing the shape) on these shapes. Instead of modifying each shape class to accommodate new operations, we can use the Visitor pattern.

Code Example (Java):

```
// Visitor Interface

interface ShapeVisitor {

    void visit(Circle circle);

    void visit(Rectangle rectangle);

}
```

```
// Element Interface

interface Shape {

    void accept(ShapeVisitor visitor);

}


// Concrete Element - Circle

class Circle implements Shape {

    private double radius;


    public Circle(double radius) {

        this.radius = radius;

    }


    public double getRadius() {

        return radius;

    }


    @Override

    public void accept(ShapeVisitor visitor) {

        visitor.visit(this);

    }

}


// Concrete Element - Rectangle
```



```
class Rectangle implements Shape {  
    private double width, height;  
  
    public Rectangle(double width, double height) {  
        this.width = width;  
        this.height = height;  
    }  
  
    public double getWidth() {  
        return width;  
    }  
  
    public double getHeight() {  
        return height;  
    }  
  
    @Override  
    public void accept(ShapeVisitor visitor) {  
        visitor.visit(this);  
    }  
}  
  
// Concrete Visitor - Area Calculation  
class AreaCalculator implements ShapeVisitor {
```

```
@Override

public void visit(Circle circle) {

    double area = Math.PI * Math.pow(circle.getRadius(), 2);

    System.out.println("Circle Area: " + area);

}


@Override

public void visit(Rectangle rectangle) {

    double area = rectangle.getWidth() * rectangle.getHeight();

    System.out.println("Rectangle Area: " + area);

}

}


// Client

public class VisitorPatternExample {

    public static void main(String[] args) {

        Shape circle = new Circle(5);

        Shape rectangle = new Rectangle(4, 6);

        ShapeVisitor areaCalculator = new AreaCalculator();

        // Calculating area of shapes

        circle.accept(areaCalculator);

        rectangle.accept(areaCalculator);

    }

}
```

```
}  
  
}
```

Output:

Circle Area: 78.53981633974483

Rectangle Area: 24.0

In this example:

- The `Shape` interface represents the **Element**.
- `Circle` and `Rectangle` are **Concrete Elements** that can accept a visitor.
- `ShapeVisitor` defines the **Visitor** interface, and `AreaCalculator` is a **Concrete Visitor** that calculates the area of the shapes.

This allows us to add new operations (e.g., calculating perimeter) by simply creating a new visitor, without modifying the existing `Circle` or `Rectangle` classes.

Key Differences:

- **Visitor Pattern:** Allows adding operations to classes without changing their structure, promoting flexibility by separating algorithms from the objects they operate on.

Both patterns help in different scenarios: **Facade** is about simplifying access to a complex system, while **Visitor** is about adding functionality to a system without changing its classes.

State Pattern

The **State** pattern is a behavioral design pattern that allows an object to change its behavior when its internal state changes. Let's look at an example of how the **State** pattern can be applied by comparing the code **before** and **after**.

Scenario:

We have a system that represents a **Vending Machine** with different states: **NoCoin**, **HasCoin**, and **SoldOut**. Depending on the state, the behavior of the vending machine changes (e.g., it accepts a coin, dispenses a product, or indicates it is sold out).

Before Applying the State Pattern:

In the "before" scenario, the vending machine checks its current state in every method using conditional statements (**if-else** or **switch**). This can lead to scattered and complex code that is hard to maintain as the number of states and actions grows.

Code Example (Java):

```
// Vending Machine without State Pattern

public class VendingMachine {

    private String state = "NoCoin"; // possible states: NoCoin,
    HasCoin, SoldOut

    public void insertCoin() {

        if (state.equals("NoCoin")) {

            System.out.println("Coin inserted.");

            state = "HasCoin";

        } else if (state.equals("HasCoin")) {

            System.out.println("Coin already inserted.");

        } else if (state.equals("SoldOut")) {

            System.out.println("Machine is sold out.");

        }

    }

}
```

```
}
```

```
public void pressButton() {  
    if (state.equals("NoCoin")) {  
        System.out.println("Insert a coin first.");  
    } else if (state.equals("HasCoin")) {  
        System.out.println("Product dispensed.");  
        state = "NoCoin";  
    } else if (state.equals("SoldOut")) {  
        System.out.println("Machine is sold out.");  
    }  
}
```

```
public void refill() {  
    if (state.equals("SoldOut")) {  
        System.out.println("Machine refilled.");  
        state = "NoCoin";  
    } else {  
        System.out.println("Machine is not empty.");  
    }  
}
```

```
public String getState() {  
    return state;  
}
```

```

    }
}

// Client

public class StatePatternBefore {

    public static void main(String[] args) {

        VendingMachine vendingMachine = new VendingMachine();

        vendingMachine.insertCoin(); // Coin inserted.

        vendingMachine.pressButton(); // Product dispensed.

        vendingMachine.insertCoin(); // Coin inserted.

        vendingMachine.pressButton(); // Product dispensed.

        vendingMachine.refill();      // Machine is not empty.

    }

}

```

Output:

```

Coin inserted.

Product dispensed.

Coin inserted.

Product dispensed.

Machine is not empty.

```

Problems:

- **Complexity:** The vending machine behavior is determined by checking the state with `if-else` conditions in each method. As the number of states and actions increases, the code becomes difficult to manage.
 - **Maintenance:** Every time a new state or action is added, multiple methods may need to be updated.
-

After Applying the State Pattern:

The **State** pattern allows us to move the state-specific behavior into separate state classes. Each state class defines how the vending machine should behave when it is in that particular state. This makes it easy to add new states or change the behavior without modifying the entire vending machine class.

Code Example (Java):

```
// State Interface
```

```
interface VendingMachineState {  
  
    void insertCoin(VendingMachine vendingMachine);  
  
    void pressButton(VendingMachine vendingMachine);  
  
    void refill(VendingMachine vendingMachine);  
  
}
```

```
// Concrete State - NoCoin
```

```
class NoCoinState implements VendingMachineState {  
  
    @Override  
  
    public void insertCoin(VendingMachine vendingMachine) {  
  
        System.out.println("Coin inserted.");  
  
        vendingMachine.setState(new HasCoinState());  
  
    }  
  
}
```

```

@Override

public void pressButton(VendingMachine vendingMachine) {

    System.out.println("Insert a coin first.");

}

@Override

public void refill(VendingMachine vendingMachine) {

    System.out.println("Machine is not empty.");

}

}

// Concrete State - HasCoin

class HasCoinState implements VendingMachineState {

    @Override

    public void insertCoin(VendingMachine vendingMachine) {

        System.out.println("Coin already inserted.");

    }

    @Override

    public void pressButton(VendingMachine vendingMachine) {

        System.out.println("Product dispensed.");

        vendingMachine.setState(new NoCoinState()); // Go back to
NoCoin state

    }

```



```
@Override

public void refill(VendingMachine vendingMachine) {

    System.out.println("Machine is not empty.");

}

}

// Concrete State - SoldOut

class SoldOutState implements VendingMachineState {

    @Override

    public void insertCoin(VendingMachine vendingMachine) {

        System.out.println("Machine is sold out.");

    }

    @Override

    public void pressButton(VendingMachine vendingMachine) {

        System.out.println("Machine is sold out.");

    }

    @Override

    public void refill(VendingMachine vendingMachine) {

        System.out.println("Machine refilled.");

        vendingMachine.setState(new NoCoinState());

    }

}
```

```
// Context Class (Vending Machine)

class VendingMachine {

    private VendingMachineState state;

    public VendingMachine() {

        state = new NoCoinState(); // Start with NoCoin state
    }

    public void setState(VendingMachineState state) {

        this.state = state;
    }

    public void insertCoin() {

        state.insertCoin(this);
    }

    public void pressButton() {

        state.pressButton(this);
    }

    public void refill() {

        state.refill(this);
    }
}
```

```
}

// Client

public class StatePatternAfter {

    public static void main(String[] args) {

        VendingMachine vendingMachine = new VendingMachine();

        vendingMachine.insertCoin(); // Coin inserted.

        vendingMachine.pressButton(); // Product dispensed.

        vendingMachine.insertCoin(); // Coin inserted.

        vendingMachine.pressButton(); // Product dispensed.

        vendingMachine.refill();      // Machine is not empty.

    }

}
```

Output:

Coin inserted.

Product dispensed.

Coin inserted.

Product dispensed.

Machine is not empty.

Benefits of the State Pattern:

1. **Simplified Code:** The logic for each state is encapsulated in separate classes. The Vending Machine class simply delegates state-specific behavior to the current state.
 2. **Easy to Extend:** Adding new states is straightforward. You can add a new state class without modifying the existing code.
 3. **Maintainability:** Each state class focuses on the specific behavior of the vending machine when it's in that state. This makes the code easier to understand and maintain.
-

Summary:

- **Before State Pattern:** The behavior of the vending machine is scattered across different methods with `if-else` conditions. This leads to complex and hard-to-maintain code as the number of states grows.
- **After State Pattern:** The state-specific behavior is encapsulated in separate state classes, and the vending machine transitions between states dynamically. This results in cleaner, more maintainable, and flexible code.